CENTRO UNIVERSITÁRIO FEI

Departamento de Ciência da Computação — CCM520: Internet das Coisas

Relatório Técnico — Projeto 2: Estufa Inteligente na Plataforma FIWARE

Integrante: Enrico N. Silva **RA: 24.226.041-2**

Professor: Prof. Dr. Rafael Gomes Alves API Key / Grupo: 0412 | Dispositivo: estufa001

Repositório GitHub: https://github.com/EnricoNSilva/Estufa-inteligente-FIWARE

1. Objetivo, Arquitetura e Identificadores do Projeto

Este projeto tem como objetivo migrar a estufa inteligente desenvolvida no Projeto 1 (baseada em ESP32 com MicroPython publicando em tópicos MQTT isolados) para uma arquitetura gerenciada por contexto utilizando a plataforma FIWARE. O circuito de hardware permanece conectado ao broker MQTT público (broker.hivemq.com:1883), mas agora integra-se ao FIWARE por meio de uma ponte (bridge) configurada no broker Mosquitto local, que filtra exclusivamente os tópicos da API key 0412. O IoT Agent JSON traduz as medidas publicadas pela ESP32 e os comandos enviados à placa para o padrão NGSI-v2 no Orion Context Broker, permitindo que aplicações externas consultem o estado atual, enviem comandos validados, recebam alertas de solo seco (via microserviço consumidor) e armazenem o histórico em banco de dados MySQL (via Cygnus) para visualização em tempo real no Grafana, sem que nenhuma dessas aplicações precise falar MQTT.

1.1 Identificadores e Mapeamento do Modelo de Dados (Grupo 0412)

Parâmetro / Componente	Valor Configurado no Projeto	Função na Arquitetura
API Key (Grupo)	0412	Isolamento lógico dos tópicos MQTT e grupo no IoT Agent
Client ID da Ponte Mosquitto	ponte_0412	Identificador único da bridge no broker.hivemq.com:1883
Client ID da ESP32	esp32_0412_estufa001	Identificador único da ESP32 no broker público
fiware-service / servicepath	fei / /estufa	Tenant (banco fei) e caminho de serviço (tabela estufa)
device_id / Entidade NGSI	estufa001 / urn:ngsi-ld:Greenhouse:001	Dispositivo no IoT Agent e entidade no Orion
Tópico de Medidas (Norte)	/json/0412/estufa001/attrs	Publicação periódica JSON da ESP32 (direção in na ponte)
Tópico de Comandos (Sul)	/0412/estufa001/cmd	Envio de comandos do IoT Agent à ESP32 (direção out)
Tópico de Resultado (Norte)	/json/0412/estufa001/cmdexe	Confirmação OK / ERRO da ESP32 (direção in na ponte)

object_id (JSON)	Atributo NGSI-v2	Tipo	Unidade / Faixa	Pino / Origem no Hardware (ESP32)
t	temperature	Number	°C	DHT22 (GPIO4) — sensor digital
rh	relativeHumidity	Number	% (0 a 100)	DHT22 (GPIO4) — sensor digital
sm	soilMoisture	Number	% (0 a 100)	ADC GPIO34 (ATTN_11DB, 0-4095 -> 0-100%)
lum	illuminance	Number	% (0 a 100)	ADC GPIO35 / LDR (ATTN_11DB, 0-4095 -> 0-100%)
rele	relayState	Number	0 ou 1	Saída digital GPIO26 (0=Desligado, 1=Ligado)
janela	windowAngle	Number	0 a 180 (°)	PWM 50 Hz GPIO27 (duty 25 a 125 -> 0° a 180°)
intervalo	samplingInterval	Number	2 a 10 (s)	Temporizador não bloqueante (ticks_ms, padrão 3 s)

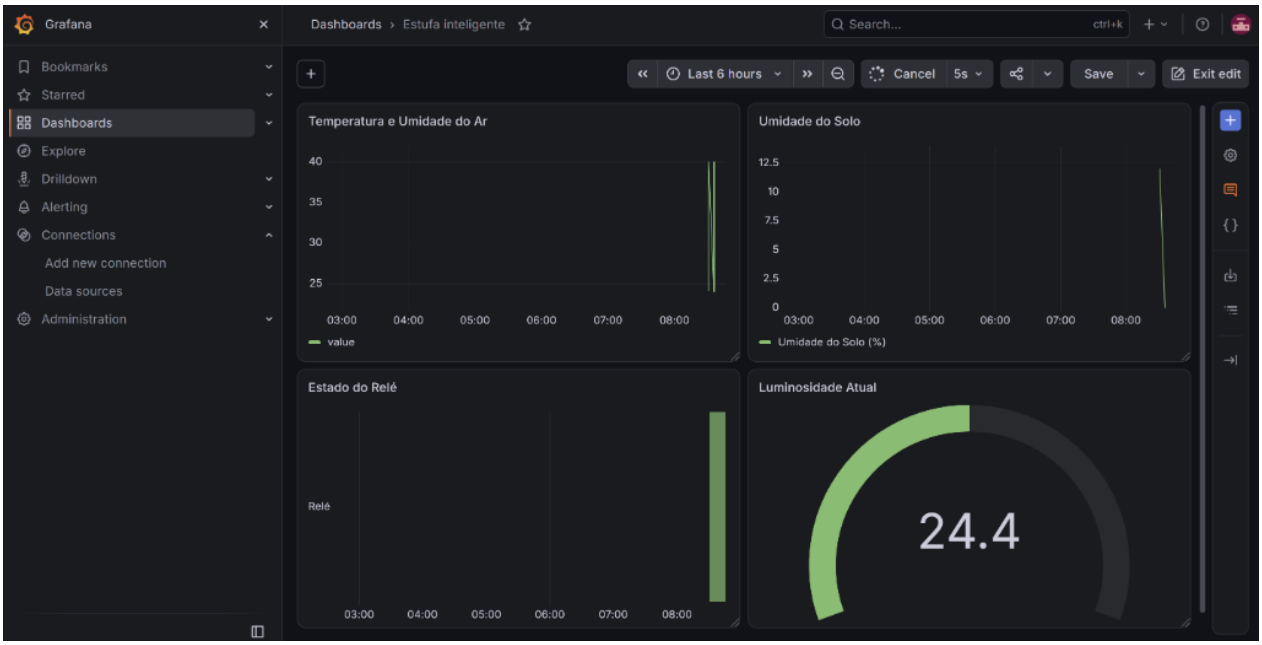
2. Decisões de Projeto, Firmware e Resultados de Verificação

1. Calibração e Conversão no Firmware: As entradas analógicas de 12 bits (GPIO34 para umidade do solo e GPIO35 para o LDR) usam atenuação ADC.ATTN_11DB (0 a 3,3 V, leituras de 0 a 4095) e são convertidas linearmente em percentual (0.0% a 100.0%) por round((leitura / 4095.0) * 100.0, 1). O servomotor em GPIO27 opera com PWM a 50 Hz, mapeando 0°–180° para duty 25–125 via int(25 + (angulo / 180.0) * 100).

2. Validação Estrita de Comandos e Prevenção de Estado Inválido: Na função executar_comando(nome, valor), além do bloco try/except (ValueError, TypeError), adicionei a verificação explícita if isinstance(valor, bool) antes de converter para int ou float, pois em Python/MicroPython bool é subclasse de int (int(True) == 1). Qualquer valor fora

de faixa (como setServo=200 ou setInterval=30) retorna imediatamente 'ERRO: <motivo>' sem alterar o estado do atuador ou a variável global.
3. Publicação Imediata Pós-Comando e Laço Não Bloqueante: Ao receber um comando em ao_receber(), o firmware publica a resposta em /json/0412/estufa001/cmdexe e, se envio_ativo for True, chama publicar_medidas() imediatamente e atualiza ultimo_envio = time.ticks_ms(). Isso faz o novo estado aparecer no Orion e no Grafana na mesma hora e evita publicações duplicadas. O laço while True usa apenas cliente.check_msg() e time.ticks_diff(), sem time.sleep() nem wait_msg().
4. Adaptação de Porta no Docker e Testes Automatizados: Como a porta 3306 do host Windows estava em uso por uma instância local do MySQL, o mapeamento externo do contêiner db-mysql foi ajustado para '3307:3306', mantendo a porta 3306 interna na rede Docker para o Cygnus e o Grafana. Na validação final pelo Runner do Postman (iniciando pela pasta 0 - Recomeçar do zero), todas as 42 requisições e 125 asserções passaram sem nenhuma falha (totalPass: 125, totalFail: 0).

2.1 Painel 'Estufa inteligente' em Operação no Grafana



3. Respostas às Questões Técnicas do Guia (R1 a R8)

R1: O que é uma assinatura compartilhada (shared subscription)? O que aconteceria com a sua estufa, em uma turma de 20 grupos, se o IoT Agent de cada grupo se conectasse direto ao broker público? Explique como a ponte filtrada pela API key evita o problema e por que cada tópico da ponte tem a direção que tem.

1. O que é uma assinatura compartilhada (shared subscription):
No protocolo MQTT, enquanto em uma assinatura convencional todos os clientes que assinam um mesmo tópico recebem uma cópia de todas as mensagens publicadas nele, uma assinatura compartilhada — escrita no formato $share/<nome-do-grupo>/<filtro-de-topico> — faz com que o broker trate todos os clientes inscritos com o mesmo <nome-do-grupo> e o mesmo <filtro> como um único assinante lógico. O broker divide (balanceia em round-robin/carga) as mensagens publicadas: cada mensagem que casa com o filtro é entregue a APENAS UM dos clientes membros daquele grupo compartilhado.

2. O que aconteceria em uma turma de 20 grupos se cada IoT Agent conectasse direto ao broker público:
O IoT Agent JSON (versão 3.14.0) não assina tópicos específicos de cada API key; ao iniciar, ele assina curingas globais dentro de um grupo compartilhado de nome fixo 'json' (codificado internamente no agente, sem variável de ambiente para alterá-lo), tais como:

- $share/json//json/+/+/attrs
- $share/json//json/+/+/cmdexe

Se os 20 grupos da turma conectassem seus IoT Agents diretamente ao broker.hivemq.com (além de outros usuários no mundo), todos os 20 IoT Agents entrariam no mesmo grupo compartilhado 'json' com exatamente o mesmo filtro curinga. Quando a minha ESP32 publicasse uma medida em /json/0412/estufa001/attrs, o tópico casaria com o curinga +/+/attrs e o broker da HiveMQ entregaria essa medida para apenas UM dos 20 IoT Agents da turma (na média, 19 em cada 20 medidas minhas iriam para os computadores dos colegas). Como os IoT Agents dos outros 19 grupos não possuem a API key 0412 provisionada, eles descartariam a minha medida como desconhecida, e o meu próprio IoT Agent (e consequentemente o meu Orion, Cygnus e Grafana) perderia ~95% ou até 100% das medidas e das respostas de comandos (cmdexe), fazendo os comandos ficarem presos em PENDING.

3. Como a ponte filtrada pela API key evita o problema e justificativa das direções dos tópicos:
Com a arquitetura em ponte (bridge), o meu IoT Agent conecta-se exclusivamente ao Mosquitto local (mosquitto:1883 na rede Docker), onde ele é o ÚNICO membro do grupo $share/json/... e, portanto, recebe 100% das mensagens locais. O Mosquitto local, por sua vez, conecta-se ao broker.hivemq.com:1883 como um cliente MQTT comum (remote_clientid ponte_0412, sem usar $share) e assina apenas os tópicos explícitos da minha API key 0412. Assim, todos os clientes normais recebem todas as mensagens daquela chave sem competir com outros grupos.
As direções configuradas no arquivo plataforma/mosquitto/mosquitto.conf seguem a origem e o destino de cada fluxo:

- topic /json/0412/estufa001/attrs in 0 (Direção IN — do broker público para o local): Quem publica as medidas dos sensores é a ESP32 (no Wokwi), que está conectada ao broker público na internet, e quem precisa recebê-las é o IoT Agent, que está conectado ao Mosquitto local. Logo, a ponte deve trazer (in) essas mensagens de fora para dentro.
- topic /json/0412/estufa001/cmdexe in 0 (Direção IN — do broker público para o local): Quem publica o resultado da execução de cada comando ('OK' ou 'ERRO: ...') também é a ESP32 no broker público, e quem precisa lê-lo para atualizar <cmd>_status e <cmd>_info no Orion é o IoT Agent no Mosquitto local. Portanto, a direção também é de entrada (in).
- topic /0412/estufa001/cmd out 0 (Direção OUT — do broker local para o público): Quem publica o comando (ex.: {"setRelay": 1}) é o IoT Agent no Mosquitto local (após receber a requisição do Orion), e quem precisa recebê-lo é a ESP32, que assina esse tópico no broker público. Logo, a ponte deve levar (out) o comando do broker local para o

público. (Além disso, separar estritamente 'in' e 'out' em vez de 'both' evita loops infinitos de republicação, já que no HiveMQ público é necessário usar try_private false).

R2: Por que a API key precisa ser única na turma? O que aconteceria se dois grupos usassem a mesma API key e o mesmo device_id?

1. Por que a API key precisa ser única na turma:
Em um broker MQTT público como o broker.hivemq.com, não há isolamento de rede nem namespaces por conta de usuário: a árvore de tópicos é única e compartilhada por todos os clientes conectados. Na convenção de tópicos do IoT Agent JSON (/json/<apikey>/<device_id>/attrs, /<apikey>/<device_id>/cmd e /json/<apikey>/<device_id>/cmdexe), o identificador do dispositivo (estufa001) é padronizado pelo roteiro para todos os grupos. Consequentemente, a API key (no meu caso, 0412) funciona como o único prefixo diferenciador ('sobrenome') capaz de separar os tópicos e os client_ids da minha estufa e da minha ponte em relação às demais estufas da turma.

2. O que aconteceria se dois grupos (Grupo A e Grupo B) usassem a mesma API key (ex.: 0412) e o mesmo device_id (estufa001):
Ocorreriam quatro falhas graves simultâneas (colisão de conexões e mistura completa de contexto):

- a) Guerra de desconexão (reconnect loop) na ponte e na ESP32 por colisão de Client ID: Pelo padrão do projeto, o client_id da ponte é ponte_<apikey> (ponte_0412) e o da ESP32 é esp32_<apikey>_<device_id> (esp32_0412_estufa001). Como a especificação MQTT exige que todo client_id seja exclusivo no broker e desconecta a sessão antiga sempre que um novo cliente conecta com o mesmo ID, as duas pontes Mosquitto e as duas ESP32 ficariam derrubando uma à outra continuamente a cada poucos segundos (gerando erros 'Socket error / closed its connection' no Mosquitto e 'ECONNRESET' na ESP32).
- b) Mistura e sobrescrita de medidas no Orion e no MySQL: Ambas as ESP32 publicariam exatamente no mesmo tópico (/json/0412/estufa001/attrs). Os dois IoT Agents receberiam pacotes de ambas as estufas como se viessem de um único dispositivo, sobrescrevendo os atributos da entidade urn:ngsi-ld:Greenhouse:001 com leituras alternadas das duas placas e corrompendo as séries temporais no MySQL/Grafana.
- c) Execução cruzada de comandos nos atuadores físicos: Quando o Grupo A enviasse um comando pelo seu Orion (ex.: setRelay = 1), o comando sairia em /0412/estufa001/cmd e seria entregue a ambas as estufas que estivessem conectadas no momento, acionando indevidamente o relé ou o servomotor do Grupo B.
- d) Conflito de respostas em cmdexe: As duas ESP32 responderiam no mesmo tópico /json/0412/estufa001/cmdexe. Se uma estufa aceitasse o comando ('OK') e a outra estivesse com firmware diferente ou respondesse fora de ordem, o estado <comando>_info no Orion seria sobrescrito pela última resposta recebida.

R3: Por que o grupo deve rejeitar dispositivos não provisionados no broker público? Dê um exemplo do que poderia acontecer sem isso.

1. Por que rejeitar dispositivos não provisionados (autoprovision: false):
No IoT Agent JSON, o parâmetro booleano autoprovision do grupo de serviço (POST /iot/groups) define como o agente reage quando recebe uma mensagem MQTT em um tópico cuja API key pertence ao grupo, mas cujo <device_id> não foi previamente cadastrado via POST /iot/devices (ou quando o grupo usa curingas como + nos tópicos). Se autoprovision estiver como true (que é o padrão caso seja omitido), o IoT Agent cria automaticamente um novo registro de dispositivo no MongoDB e uma nova entidade no Orion usando os nomes brutos das chaves do JSON como nomes de atributos. Em um broker público sem autenticação, onde qualquer pessoa pode publicar em qualquer tópico, manter autoprovision: false (junto com o filtro estrito de device_id na ponte) garante que o IoT Agent só processe medidas de dispositivos explicitamente autorizados e mapeados pelo grupo.

2. Exemplo prático do que poderia acontecer sem isso:
Suponha que a ponte usasse um curinga de dispositivo (/json/0412/+/attrs) e o grupo 0412 estivesse com autoprovision: true:

- Caso 1 (Erro de digitação / Poluição de contexto e banco): Se durante um teste no HiveMQ WebSocket Client ou

no firmware eu digitasse por engano o tópico /json/0412/estufa01/attrs (faltando um zero) com o payload {"t": 25.0, "sm": 20.0}, o IoT Agent não rejeitaria a mensagem; em vez disso, criaria silenciosamente uma entidade fantasma 'Greenhouse:estufa01' no Orion com atributos não traduzidos ('t' e 'sm' em vez de 'temperature' e 'soilMoisture'). Como a subscrição do Cygnus observa idPattern: '.*' para o tipo Greenhouse, essa entidade fantasma começaria a disparar notificações e poluir a tabela estufa no MySQL.

• Caso 2 (Ataque de exaustão / Injeção por terceiros no broker público): Um script malicioso conectado ao broker.hivemq.com poderia publicar milhares de mensagens em /json/0412/falso_001/attrs, /json/0412/falso_002/attrs, ..., /json/0412/falso_9999/attrs. Com autoprovision: true, o IoT Agent criaria milhares de dispositivos falsos no MongoDB e milhares de entidades Greenhouse no Orion, disparando enxurradas de notificações para o Cygnus e esgotando o armazenamento do MySQL e a memória do Orion. Com autoprovision: false, o IoT Agent descarta imediatamente qualquer device_id não provisionado (logando apenas MEASURES-004: Device not found).

R4: Descreva o caminho de uma medida, da ESP32 até o painel do Grafana: componente, protocolo, porta e formato do dado em cada salto.

Quando o temporizador não bloqueante da ESP32 atinge o intervalo de coleta (time.ticks_diff(agora, ultimo_envio) >= intervalo), a função publicar_medidas() lê o DHT22, os dois canais ADC e o estado dos atuadores, iniciando o tráfego Norte (dispositivo -> plataforma) através dos seguintes 7 saltos:

Salto	Origem -> Destino	Protocolo e Porta	Endpoint / Tópico / Tabela e Formato do Dado
1	ESP32 -> Broker Público	MQTT / TCP :1883	PUBLISH em /json/0412/estufa001/attrs (JSON com object_id curtos)
2	Broker Público -> Mosquitto	MQTT Bridge :1883	Ponte 'ponte_0412' (regra in 0) repassa o mesmo tópico e payload
3	Mosquitto -> IoT Agent JSON	MQTT / TCP :1883	Assinatura $share/json//json/+/+/attrs entrega ao IoT Agent
4	IoT Agent -> Orion (+Mongo)	HTTP / TCP :1026	POST /v2/op/update (NGSI-v2) traduzindo object_id -> nome NGSI
5	Orion -> FIWARE Cygnus	HTTP / TCP :5050	POST http://cygnus:5050/notify (JSON formato NGSI-v2 normalized)
6	Cygnus -> Banco MySQL	MySQL / TCP :3306	INSERT na tabela fei.estufa (modelo dm-by-service-path, 7 linhas)
7	Grafana -> Banco MySQL	MySQL / TCP :3306	SELECT com CAST e $__timeFilter (usuário grafana) a cada 5 s

Detalhamento técnico dos formatos de dados em cada salto:

• Saltos 1, 2 e 3 (ESP32 -> HiveMQ -> Ponte Mosquitto -> IoT Agent via MQTT :1883): A ESP32 publica no tópico /json/0412/estufa001/attrs um JSON compacto com chaves curtas (object_id) e valores estritamente numéricos: {"t": 24.0, "rh": 40.0, "sm": 100.0, "lum": 24.4, "rele": 1, "janela": 90, "intervalo": 5}

• Salto 4 (IoT Agent -> Orion Context Broker via HTTP :1026): O IoT Agent consulta o provisionamento de estufa001, traduz as chaves curtas para os nomes NGSI-v2 (t -> temperature, rh -> relativeHumidity, sm -> soilMoisture, lum -> illuminance, rele -> relayState, janela -> windowAngle, intervalo -> samplingInterval), injeta o atributo e metadado TimeInstant (pois IOTA_TIMESTAMP=true) e envia um POST http://orion:1026/v2/op/update com cabeçalhos fiware-service: fei e fiware-servicepath: /estufa. O Orion atualiza o documento da entidade urn:ngsi-ld:Greenhouse:001 no MongoDB (mongo-db:27017).

• Salto 5 (Orion -> Cygnus via HTTP :5050): Ao detectar a atualização dos atributos da entidade Greenhouse, o Orion dispara a subscrição de histórico (6af481bb18c64854510f7d82), enviando um POST http://cygnus:5050/notify no formato normalized contendo o array data[] com a entidade e os 7 atributos de medida, cada um estruturado como {"type": "Number", "value": <valor>, "metadata": {"TimeInstant": {"type": "DateTime", "value": "..."}}}.

• Salto 6 (Cygnus -> MySQL via TCP :3306 interno / :3307 no host): Como o Cygnus está configurado com CYGNUS_MYSQL_DATA_MODEL=dm-by-service-path e CYGNUS_MYSQL_ATTR_PERSISTENCE=row, ele usa o cabeçalho fiware-service: fei para selecionar o schema 'fei' e fiware-servicepath: /estufa para selecionar a tabela 'estufa'. Para uma única medida com 7 atributos, ele executa 7 inserções (uma linha por atributo) preenchendo as colunas texto recvTimeTs, recvTime, fiwareServicePath, entityId, entityType, attrName, attrType, attrValue e attrMd.

• Salto 7 (Grafana -> MySQL via TCP :3306 interno e HTTP :3000 para o navegador): A cada 5 segundos (autoRefresh: '5s'), o Grafana (localhost:3000) executa as queries SQL sobre mysql-db:3306 com o usuário somente

leitura 'grafana', convertendo as colunas texto com CAST(recvTime AS DATETIME(3)) AS time e CAST(attrValue AS DECIMAL(6,1)) para renderizar os gráficos Time series, State timeline e Gauge no navegador.

R5: Descreva o caminho de um setRelay, do Postman até o relé e de volta. Quando o _status fica PENDING e quando vira OK? Por que OK não garante que a estufa aceitou o comando (requisição 3.7)?

1. Por que a entidade já existe no Orion logo após o provisionamento, mas apenas com os comandos:
Ao executar a requisição 2.2 (POST /iot/devices), o IoT Agent cria a entidade urn:ngsi-ld:Greenhouse:001 no Orion inicializando os atributos de comando (setRelay, setServo, setInterval, setSending) e registra a si mesmo como Context Provider (visível em GET /v2/registrations, apontando para http://iot-agent:4041/op/update) para esses 4 atributos. Os 7 atributos de medida só passam a existir na entidade quando a primeira medida é publicada pela estufa.

2. Caminho completo do comando setRelay (Tráfego Sul na ida e Norte na volta):

- Passo 1 (Postman -> Orion, HTTP :1026): A aplicação/Postman envia PATCH http://localhost:1026/v2/entities/urn:ngsi-ld:Greenhouse:001/attrs com os cabeçalhos fiware-service: fei e fiware-servicepath: /estufa e o corpo JSON: {"setRelay": {"type": "command", "value": 1}}.
- Passo 2 (Orion -> IoT Agent JSON, HTTP :4041): O Orion verifica em sua tabela de registros (registrations) quem é o provedor responsável pelo atributo setRelay daquela entidade, encaminha a requisição via POST http://iot-agent:4041/op/update para a North Port do IoT Agent e responde imediatamente 204 No Content ao Postman.
- Passo 3 (Quando setRelay_status fica PENDING — IoT Agent -> Orion e Mosquitto): Assim que o IoT Agent recebe o comando do Orion, ANTES mesmo de a ESP32 responder, o IoT Agent atualiza no Orion o atributo setRelay_status = "PENDING" (indicando que o comando foi despachado e aguarda confirmação) e publica via MQTT (:1883) no Mosquitto local, no tópico /0412/estufa001/cmd, o payload JSON: {"setRelay": 1}.
- Passo 4 (Mosquitto Local -> Ponte -> Broker Público -> ESP32): A ponte Mosquitto (ponte_0412), pela regra 'topic /0412/estufa001/cmd out 0', encaminha a mensagem para o broker.hivemq.com:1883, que a entrega à ESP32 (inscrita nesse tópico). No laço principal, cliente.check_msg() invoca o callback ao_receber(topico, mensagem).
- Passo 5 (Execução no Hardware e Resposta da ESP32 -> Broker Público -> Ponte -> Mosquitto Local): A função executar_comando("setRelay", 1) valida que 1 pertence a (0, 1), aciona o pino físico GPIO26 com rele.value(1) e retorna "OK". O callback ao_receber() publica então no tópico de resultado /json/0412/estufa001/cmdexe o JSON {"setRelay": "OK"} (e em seguida chama publicar_medidas() enviando "rele": 1 em /json/0412/estufa001/attrs). A ponte Mosquitto, pela regra 'in 0' em cmdexe, traz a resposta para o Mosquitto local.
- Passo 6 (Quando setRelay_status vira OK — IoT Agent -> Orion): O IoT Agent (inscrito em $share/json//json/+/+/cmdexe) recebe {"setRelay": "OK"}, identifica a chave "setRelay" e envia uma atualização NGSI-v2 ao Orion (POST /v2/op/update) mudando setRelay_status de "PENDING" para "OK" e gravando o texto retornado em setRelay_info = "OK".

3. Por que <comando>_status = 'OK' NÃO garante que a estufa aceitou ou executou o comando (Caso da Requisição 3.7):
Para o IoT Agent JSON e o Orion, o atributo <comando>_status reflete exclusivamente o estado da TRANSAÇÃO DE TRANSPORTE (entrega e resposta do protocolo): 'PENDING' significa que o dispositivo ainda não respondeu, e 'OK' significa apenas que o dispositivo recebeu a mensagem MQTT e devolveu uma resposta válida no tópico cmdexe com a chave do comando. O IoT Agent não interpreta a semântica do texto devolvido pela ESP32.
Isso ficou comprovado na prática nas requisições 3.6 e 3.7 do projeto: quando enviei o comando inválido setServo = 200 (fora da faixa de 0° a 180°), a ESP32 recebeu a mensagem, recusou a alteração do servomotor (mantendo windowAngle em 90°) e respondeu no tópico /json/0412/estufa001/cmdexe o payload {"setServo": "ERRO: valor deve ser de 0 a 180"}. Como a estufa respondeu ao comando, o Orion passou a exibir:

- setServo_status: "OK" (a comunicação de comando e resposta concluiu com sucesso)
- setServo_info: "ERRO: valor deve ser de 0 a 180" (o comando foi rejeitado pela lógica de validação da estufa)
- windowAngle: 90 (o estado físico anterior foi preservado).

Portanto, qualquer aplicação consumidora precisa inspecionar <comando>_info (e o atributo de estado correspondente), nunca apenas <comando>_status.

R6: Por que setSending = 0 não pode desligar a escuta de comandos?

O comando setSending = 0 tem como função pausar apenas a publicação periódica de medidas de sensores pela ESP32 (controlada pela flag booleana envio_ativo = False no firmware), mas jamais pode encerrar a conexão MQTT nem parar de chamar cliente.check_msg() no laço principal pelos seguintes motivos técnicos:

- 1. Perda irreversível de controle remoto (Deadlock operacional / Dispositivo 'surdo'): Se ao receber setSending = 0 a ESP32 parasse de escutar o tópico /0412/estufa001/cmd, ela nunca mais conseguiria receber o comando setSending = 1 (requisição 3.14) para reativar o envio das medidas! A única forma de recuperar a estufa seria ir fisicamente até a placa apertar o botão de Reset (EN) ou cortar a energia.
- 2. Necessidade contínua de atuação (irrigação e ventilação) independente da telemetria: Mesmo quando o produtor ou sistema deseja pausar temporariamente a coleta periódica para economizar banda/armazenamento, os atuadores críticos da estufa (ligar/desligar o relé via setRelay, abrir/fechar a janela via setServo ou alterar a taxa via setInterval) precisam continuar operando e respondendo no tópico cmdexe para não deixar os comandos em estado PENDING no Orion.

Comprovação real obtida nos testes 3.12 a 3.15 do projeto:
No firmware (firmware/main_esqueleto.py), o laço principal executa cliente.check_msg() incondicionalmente em todas as iterações, enquanto publicar_medidas() fica protegido pela condição 'if envio_ativo and time.ticks_diff(agora, ultimo_envio) >= intervalo:'.

- Ao enviar setSending = 0 (requisição 3.12), a verificação 3.13 leu o metadado TimeInstant de temperature no Orion (valor real obtido: "2026-11-11T11:21:34.758Z"), aguardou 12 segundos (mais que o dobro do intervalo de 5 s configurado em 3.8) e consultou novamente, obtendo exatamente o mesmo TimeInstant ("2026-11-11T11:21:34.758Z"), provando que nenhuma medida foi enviada.
- Em seguida, como a escuta de comandos permaneceu ativa, a requisição 3.14 enviou setSending = 1 pelo Orion, a ESP32 recebeu o comando imediatamente, respondeu {"setSending": "OK"} e retomou as publicações, gerando um novo TimeInstant recente na requisição 3.15 ("2026-11-11T11:22:34.701Z", com idade inferior a 8 s).

R7: O que o Orion guarda e o que o MySQL guarda? Por que a subscrição do Cygnus usa normalized e a do alerta usa keyValues?

1. Diferença entre o que o Orion guarda e o que o MySQL guarda:

- Orion Context Broker (apoiado pelo MongoDB): Guarda apenas o ESTADO DE CONTEXTO ATUAL (último valor conhecido) de cada entidade (urn:ngsi-ld:Greenhouse:001). Quando uma nova medida de temperatura chega da ESP32 (ex.: 24.5 °C), o Orion sobrescreve o valor anterior (ex.: 24.0 °C) e atualiza seu metadado TimeInstant. O Orion não mantém série histórica: não é possível consultar nele qual era a temperatura da estufa há 2 horas.
- MySQL (alimentado pelo FIWARE Cygnus): Guarda o HISTÓRICO TEMPORAL COMPLETO (série histórica imutável) de todas as medidas notificadas ao longo do tempo. A cada notificação recebida pelo Cygnus, novas linhas são acrescentadas (INSERT) na tabela 'estufa' do banco 'fei' (7 linhas por ciclo de leitura, uma para cada atributo de medida, identificadas por recvTimeTs/recvTime, attrName e attrValue), permitindo consultas agregadas e gráficos de evolução temporal no Grafana.

2. Por que a subscrição do Cygnus (4.1) usa 'normalized' e a do alerta de solo seco (4.4) usa 'keyValues':

- Por que 'normalized' para o Cygnus (http://cygnus:5050/notify): No formato NGSI-v2 normalized (padrão completo do Orion), cada atributo da notificação é um objeto estruturado contendo explicitamente 'type', 'value' e 'metadata':
  "temperature": {"type": "Number", "value": 24.0, "metadata": {"TimeInstant": {"type": "DateTime", "value": "..."}}}
  O conector FIWARE Cygnus exige obrigatoriamente esse formato porque o seu parser interno precisa ler o campo 'type' (para gravar a coluna attrType no MySQL) e o objeto 'metadata' (para gravar a coluna attrMd e extrair timestamps). Se a subscrição enviasse keyValues para o Cygnus, ele rejeitaria a notificação com erro HTTP 400 (Bad Request) e nada seria gravado no MySQL. Além disso, a subscrição 4.1 filtra explicitamente apenas os 7

atributos de medida para não gravar atributos estáticos ou de comando na tabela histórica.

• Por que 'keyValues' para o Alerta no serviço externo (http://consumidor:3000/alerta-estufa): No formato simplificado keyValues, cada atributo vai diretamente como um par chave-valor primitivo no JSON:

{"id": "urn:ngsi-ld:Greenhouse:001", "type": "Greenhouse", "soilMoisture": 12, "relayState": 1, "TimeInstant": "2026-11-11T11:30:40.477Z"}

O microserviço externo (consumidor.py) é uma aplicação leve de negócio que só precisa ler diretamente dado['soilMoisture'] e dado['relayState'] para exibir ou processar o alerta quando soilMoisture < 30, sem precisar navegar em sub-dicionários .value/.metadata. Na subscrição 4.4 também foi configurado throttling: 10 (segundos) porque cada medida da ESP32 atualiza o metadado TimeInstant de soilMoisture; sem o throttling, o Orion dispararia 20 alertas por minuto (a cada 3 s) enquanto o solo permanecesse seco.

R8: Qualquer pessoa conectada ao broker público pode ler e publicar nos seus tópicos. Descreva dois ataques possíveis à sua estufa e duas medidas que os evitariam em um sistema real.

Como o broker.hivemq.com na porta 1883 opera com MQTT sobre TCP em texto puro, sem autenticação de clientes e sem controle de acesso a tópicos (ACL), qualquer atacante conectado ao broker pode assinar '#' para descobrir os tópicos /json/0412/estufa001/attrs e /0412/estufa001/cmd e executar ataques que violam a Confidencialidade, a Integridade e a Disponibilidade do sistema:

Dois ataques possíveis à estufa no broker público:

• Ataque 1 — Injeção de Comandos Maliciosos e Falsificação de Medidas (Violação de Integridade): Um atacante que descobre a API key 0412 (assinando '#') pode publicar diretamente no tópico /0412/estufa001/cmd o comando {"setRelay": 1} (inundando a plantação com irrigação contínua) ou {"setSending": 0} (cegando o monitoramento da plataforma). Simultaneamente, mesmo com setSending = 0 na ESP32 real, o atacante pode publicar medidas falsas em /json/0412/estufa001/attrs como {"t": 24.0, "rh": 60.0, "sm": 85.0, "lum": 50.0, "rele": 0, "janela": 0, "intervalo": 3}. O IoT Agent e o Orion acreditarão que o solo está úmido (85%) e o relé desligado, impedindo o disparo do alerta de solo seco (soilMoisture < 30) enquanto a estufa real está sofrendo dano físico.

• Ataque 2 — Negação de Serviço (DoS) por Sequestro de Client ID (Session Takeover — Violação de Disponibilidade): O atacante pode conectar um script ao broker.hivemq.com:1883 usando exatamente o mesmo client_id da minha ponte (ponte_0412) ou da minha ESP32 (esp32_0412_estufa001). Pela especificação do protocolo MQTT 3.1.1, sempre que um segundo cliente conecta com um client_id já ativo, o broker derruba imediatamente a conexão TCP do cliente legítimo. Quando o Mosquitto ou a ESP32 tentar reconectar, o script do atacante conecta novamente em loop, mantendo a ponte e a estufa permanentemente desconectadas e indisponíveis. (Adicionalmente, como a porta 1883 não usa criptografia, um atacante na rede local pode interceptar todo o tráfego em texto puro — violação de Confidencialidade).

Duas medidas de segurança que evitariam esses ataques em um sistema real:

• Medida 1 — Broker MQTT Privado/Gerenciado com Autenticação Forte e Controle de Acesso por Tópico (ACL): Substituir o broker público aberto por um broker próprio ou gerenciado que exija autenticação individual por cliente (usuário/senha exclusivos ou certificados digitais X.509 via mTLS) e aplique listas de controle de acesso (ACL) baseadas na identidade autenticada: (i) a credencial da ESP32 só tem permissão de escrita (PUBLISH) em /json/0412/estufa001/attrs e /json/0412/estufa001/cmdexe e leitura (SUBSCRIBE) em /0412/estufa001/cmd; (ii) a credencial do IoT Agent tem o inverso; e (iii) o broker vincula cada client_id exclusivamente ao seu usuário/certificado autenticado. Isso impede completamente tanto a publicação de comandos/medidas falsas por terceiros (Ataque 1, garantindo Integridade) quanto a derrubada de sessão por colisão de client_id ou leitura por curinga '#' (Ataque 2, garantindo Disponibilidade e Confidencialidade no broker).

• Medida 2 — Criptografia de Transporte com TLS (Porta 8883) combinada com Proteção das Portas HTTP do FIWARE: Migrar toda a comunicação MQTT (da ESP32 e do broker) da porta 1883 para a porta 8883 sobre TLS (Transport Layer Security) com validação do certificado do servidor. O TLS criptografa o tráfego fim a fim na rede, impedindo que atacantes em redes intermediárias (ex.: Wi-Fi compartilhado) façam sniffing da API key, das credenciais ou adulterem pacotes em trânsito (Man-in-the-Middle). Ressalta-se que o TLS sozinho em um broker sem

autenticação protege o canal de rede (Confidencialidade em trânsito), mas precisa estar aliado à Medida 1 (autenticação + ACL no broker) e ao isolamento das portas HTTP do Orion (:1026) e do IoT Agent (:4041/:7896) via firewall/API Gateway (ex.: FIWARE PEP Proxy Wilma + Keyrock) para garantir segurança completa de ponta a ponta.